Computers & Security 148 (2025) 104140

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

PenGym: Realistic training environment for reinforcement learning pentesting agents

Huynh Phuong Thanh Nguyen [a,*], Kento Hasegawa [b], Kazuhide Fukushima [b], Razvan Beuran [a]

[a] Japan Advanced Institute of Science and Technology, Japan
[b] KDDI Research, Inc., Japan

ARTICLE INFO

Keywords:
Penetration testing
Reinforcement learning
Agent training environment
Realistic environment
Cyber range
Cybersecurity

ABSTRACT

Penetration testing, or pentesting, refers to assessing network system security by trying to identify and exploit any existing vulnerabilities. Reinforcement Learning (RL) has recently become an effective method for creating autonomous pentesting agents. However, RL agents are typically trained in a simulated network environment. This can be challenging when deploying them in a real network infrastructure due to the lack of realism of the simulation-trained agents.

In this paper, we present PenGym, a framework for training pentesting RL agents in realistic network environments. The most significant features of PenGym are its support for real pentesting actions, full automation of the network environment creation, and good execution performance. The results of our experiments demonstrated the advantages and effectiveness of using PenGym as a realistic training environment in comparison with a simulation approach (NASim). For the largest scenario, agents trained in the original NASim environment behaved poorly when tested in a real environment, having a high failure rate. In contrast, agents trained in PenGym successfully reached the pentesting goal in all our trials. Even after fixing logical modeling issues in simulation to create the revised version NASim(rev.), experiment results with the largest scenario indicated that agents trained in PenGym slightly outperformed, and were more stable, than those trained in NASim(rev.). Thus, the average number of steps required to reach the pentesting goal was 1.4 to 8 steps better for PenGym. Consequently, PenGym provides a reliable and realistic training environment for pentesting RL agents, eliminating the need to model agent actions via simulation.

1. Introduction

Ensuring the security of network systems and infrastructure is a critical aspect of cybersecurity (Zennaro and Erdődi, 2023; Li et al., 2023). Penetration testing, or pentesting, is widely considered to be the most effective method for evaluating network security (Stefinko et al., 2016). This approach involves authorized network attacks on computer systems to identify vulnerabilities and assess their overall security characteristics. However, traditional pentesting, which is conducted manually, poses several challenges, as it is time-consuming and requires technical security skills.

Recently, Reinforcement Learning (RL) appeared as a promising solution for bridging this gap by automating pentesting. RL pentesting agents are designed to mimic the actions of human pentesters but with enhanced speed, scale, and precision. Many simulation environments have been introduced to train these agents, which utilize logical modeling for the agents and network environments. Their heavy reliance on predefined constants and probabilistic values for agent actions and environment states leads to potential inaccuracies in replicating real-world behavior due to unexpected factors, decreasing agent accuracy and performance. Additionally, the simulated network may not accurately represent the configuration and topology of an actual network.

In this paper, we present an effective and reliable realistic training framework for RL pentesting agents, named PenGym. This research extends the preliminary implementation presented in Nguyen. et al. (2024), our initial solution for a realistic training environment that enables RL agents to execute actual actions on hosts in a network environment (a.k.a. cyber range) dedicated to agent training. The applicability of PenGym was enhanced in several ways, as follows: (i) Support for more complex pentesting scenarios by implementing additional actions; (ii) Full automation of cyber range creation by leveraging the tools presented in Beuran et al. (2018); (iii) Improved execution performance by applying optimization mechanisms to minimize the execution time for each action, as well as the training

* Corresponding author.
E-mail address: thanh.nguyen@jaist.ac.jp (H.P.T. Nguyen).

https://doi.org/10.1016/j.cose.2024.104140
Received 24 July 2024; Received in revised form 21 September 2024; Accepted 27 September 2024
Available online 5 October 2024
0167-4048/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

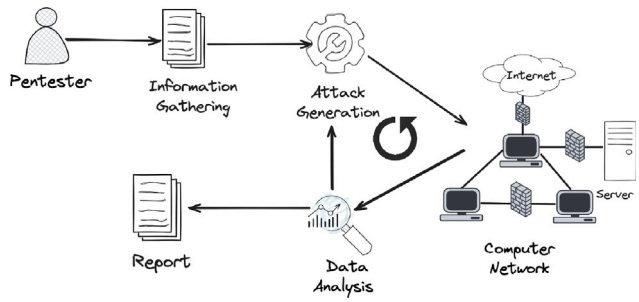


Fig. 1. The process of executing pentesting in practice.

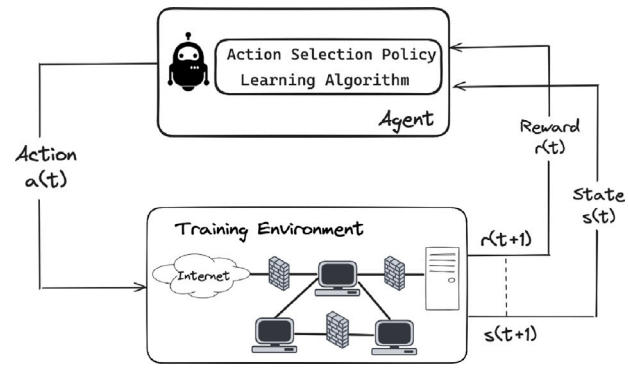


Fig. 2. The procedure of using RL for pentesting.

time. Moreover, several additional experiments with more complex and challenging pentesting scenarios were conducted to demonstrate the usefulness and effectiveness of using PenGym compared to simulation, particularly the simulation environment named NASim (Schwartz and Kurniawati, 2019).

The contributions of this paper cover a wide variety of aspects, as summarized below:

1. Discuss the advantages and motivations of using RL techniques for pentesting purposes and a realistic training environment approach for pentesting agents, as opposed to simulation (Section 2).
2. Enhance PenGym with support for a wider range of actions, action execution and training time optimization, and automated cyber range creation (Sections 4.2, 4.3, and 4.4, respectively).
3. Validate PenGym functionality and cyber range creation in various pentesting scenarios (Section 5).
4. Present additional experiments that demonstrate the effectiveness of using PenGym compared to NASim in complex pentesting scenarios (Section 6).
5. Propose a revision of NASim, named NASim(rev.), that fixes all the unrealistic modeling issues that we identified (Section 7).

This paper is organized as follows. Section 2 clarifies the motivation for using RL and a realistic environment for pentesting. Related works are discussed in Section 3. Section 4 provides an overview of PenGym, followed by a detailed description of action implementation, performance optimization, and automated cyber range creation. The results of the functionality validation and full-scale experiments are presented in Sections 5 and 6, respectively. The paper ends with a discussion, conclusion, and references.

2. Background and motivation

From a practical perspective, pentesting is a multi-stage process that demands high competence and technical expertise due to the complexity of computer networks (Ghanem and Chen, 2019). As illustrated in Fig. 1, a pentester needs to gather information about the target system and determine the type of attack to execute. The target system then responds to their attack action. After that, the pentester analyzes this response to choose an appropriate next step. This continues until testing is completed and a report is created.

As alluded to earlier, pentesting is a process that relies on learning from the current state or observation of an action to decide the next step. Without previous experience, a brute force approach is used to explore all possible testing methods for the system. However, this approach can become overwhelming as the network grows in complexity and size, and it is not feasible for real-world networks.

Cybersecurity researchers have recently identified the need for an intelligent pentesting framework to support human experts by handling high-demand tasks, including information gathering, vulnerability assessment, and exploitation (Sarraute et al., 2012; Ghanem and Chen, 2018). RL is considered a promising approach in this area, as it allows an automated pentesting system to operate and gain skills gradually, similar to a real tester. Moreover, RL algorithms have been proven to work particularly well for cybersecurity (Nguyen and Reddi, 2021). In particular, RL agents can leverage their exploration capabilities to discover previously unknown attack scenarios.

Theoretically, an RL agent interacts with its environment by performing actions and observing the results. Each action results in positive or negative feedback, depending on its outcome. Through trial and error, the agent learns an optimal policy to solve sequential decision-making problems (Sutton and Barto, 2018).

Fig. 2 displays the procedure of using RL for pentesting. An RL agent interacts with a training environment over time. At each time step t, the agent receives a state $s(t)$ in a state space S that details the environment, such as network topology, host configuration, etc. It then selects an action $a(t)$ from an action space A, according to a learning policy that maps the state $s(t)$ to an action $a(t)$. After acting, the agent receives a $r(t)$ reward and transitions to the next state $s(t+1)$. The reward $r(t)$ is based on the quality of the results from the action. The agent aims to optimize the learning policy by selecting the most suitable action at each step to achieve its objective, which is for pentesting purposes to gain unauthorized access to the system.

Several simulation frameworks have been proposed in recent years to create a training environment for RL pentesting agents. However, these frameworks may encounter functional issues caused by the modeling of agent actions and the required environment state.

The specific issues that we have identified for NASim (Schwartz and Kurniawati, 2019), which is one of the most recent environments of this type, will be described in detail in Section 7. These problems indicate that the logic in NASim is incorrect for some actions, and the simulation mechanisms do not cover all situations, such as the lack of firewalls in some attacks. This highlights the need for a realistic environment for training pentesting RL agents where observations are derived from real actions executed in an actual cyber range.

Furthermore, preparing the training environment in existing frameworks presents challenges due to the manual cyber range creation inside this environment. Creating complex networks involving numerous hosts with multiple services or processes is time-consuming and difficult. This issue motivated us to automate the creation of the cyber range.

3. Related work

A growing number of research works have been published on automated pentesting topics by creating a training environment for RL pentesting agents. Table 1 provides a summary of the most representative studies in this context and their characteristics when compared to PenGym. All approaches are compared based on their abstraction level and execution environment features.

Table 1
Comparison of related frameworks from abstraction level and environment feature perspectives.

	Small world (Furfaro et al., 2018)	BRAWL (The MITRE Corporation, 2018)	GALAXY (Schoonover et al., 2018)	CyGIL (Li et al., 2022)	Microsoft CBS (Microsoft Defender Research Team, 2021)	CybORG (Standen et al., 2021)	FARLAND (Molina-Markham et al., 2021)	NASim (Schwartz and Kurniawati, 2019)	NASimEmu (Janisch et al., 2024)	PenGym
Abstraction Level										
Simulation Based					✓	✓	✓	✓	✓	✓
Real Hosts/Emulation	✓	✓	✓	✓		✓	✓		✓	✓
Real Network Topology	✓	✓	✓	✓		✓	✓		✓	✓
Real Actions	✓	✓	✓	✓		✓	✓		✓	✓
Real Observations	✓	✓	✓	✓		✓	✓		✓	✓
Designed for RL				✓	✓	✓	✓	✓	✓	✓
Host-Based Exploitation	✓	✓	✓	✓	✓	✓		✓	✓	✓
Network-Based Exploitation	✓	✓	✓	✓			✓	✓	✓	✓
Environment Features										
Firewalls	✓							✓		✓
Network Scanning	✓	✓	✓	✓			✓	✓	✓	✓
Host Scanning (OS Scan, Process Scan, Service Scan)								✓		✓
Exploits	✓	✓	✓	✓	✓	✓		✓	✓	✓
Privilege Escalation	✓		✓	✓	✓	✓		✓	✓	✓

3.1. RL training environment

The research works related to pentesting frameworks include environments for security education (SmallWorld (Furfaro et al., 2018) and BRAWL (The MITRE Corporation, 2018)), as well as pentesting training environments using AI assistants (Pozdniakov et al., 2020; Chaudhary et al., 2020) and Markov decision processes (Ghanem and Chen, 2018). However, these frameworks lack detailed information on the RL training environment and have limited actions and scenarios.

Several simulation and emulation approaches have been presented, each having certain limitations when compared to PenGym. CyberBattleSim (CBS) from Microsoft (Microsoft Defender Research Team, 2021) and NASim (Schwartz and Kurniawati, 2019) are examples of the simulation approach. They depict a network environment based on predefined scenarios, including traffic rules, host connection, and configurations. A *reality gap* occurs due to the assumptions regarding agent actions, with their success being determined based on logical and probability conditions.

Regarding the emulation approach, CyGIL (Li et al., 2022) provides an RL training testbed with network and host-level actions. However, it omits network traffic rules and host scanning phases, creating potentially unrealistic conditions. CybORG (Standen et al., 2021) is another work for training agents in adversarial scenarios, focusing on host-based exploitation actions but neglecting network traffic discovery.

FARLAND (Molina-Markham et al., 2021) provides a training environment for attack and defense agents through simulation and emulation. However, the red agents use probabilistic factors rather than RL algorithms to determine their actions. GALAXY (Schoonover et al., 2018) is an emulated cybersecurity environment for training red agents using evolutionary algorithms. However, these frameworks lack RL capabilities, which are essential for creating intelligent pentesting agents.

NASimEmu (Janisch et al., 2024) is one of the latest frameworks for RL pentesting agents, which are trained in simulation and then deployed in an emulated environment to verify the realism of the abstraction used. NASimEmu makes it possible to train the agents simultaneously in multiple scenarios randomly selected after each episode. This is because its goal is to develop general agents that can transfer to novel scenarios unseen during training. The simulation mode, revised from the original NASim, changed the scenario implementation to allow dynamic scenarios and fixed certain objects, such as Host Vector and Observation, to make possible the multiple scenario simultaneous training approach. For this purpose, the size of the Host Vector is unified across all scenarios. Still, the Observation size varies depending on the number of visible hosts and total hosts in the scenario instance. In the emulation part, the agent's actions are translated into predefined Metasploit modules to execute actual actions. However, the emulation action space lacks Process Scan and Privilege Escalation actions. This limits the available actions, leaving the agent with only one option to gain privileged permissions on the host, Exploits.

The experiments done with NASimEmu demonstrated that a simulation-trained agent can be deployed in emulation. However, the scenarios used in those experiments were simple, with only one or two subnets and a single privilege escalation action. Moreover, the paper states that the firewall blocks or allows all traffic between subnets, which does not reflect conditions in real networks. In more complex scenarios, where each host may run multiple services but only a few are allowed through firewalls, unexpected situations in the simulation environment may affect agent performance, as discussed in Section 7.

In summary, all the frameworks mentioned above have their limitations. Simulation environments designed for autonomous RL pentesting agents have a fast execution time and require limited resources. However, they contain modeled elements that may lead to unexpected issues, resulting in unrealistic results and affecting the applicability of a trained agent on actual network infrastructure. Other environments use an emulation approach to bridge the *reality gap* of simulation. Some support only host-based exploitation or network-based discovery actions. Moreover, some systems fail to consider the host configuration before attempting an attack or do not support external firewalls.

3.2. Cyber range creation for pentesting

The cyber range is a key component of the training environment, containing a set of hosts that are connected following the network topology from a specific pentesting scenario. In addition, the host services, processes, vulnerabilities, and traffic rules are configured within the cyber range according to the host configuration specified in this scenario. Previous studies have noted support for full virtualization using KVM but lack automatic virtual interfaces and bridge setups (Schoonover et al., 2018; Standen et al., 2021). Other frameworks mentioned that the included VMs and vulnerable applications are set up manually in advance (Furfaro et al., 2018).

This research leverages the CyRIS module described in Beuran et al. (2018) to enable the automatic creation of the cyber range environment. For this purpose, a PenGym module was developed to convert the pentesting scenario into a CyRIS description. Our implementation covers host configuration and network connection, including bridges and virtual interfaces.

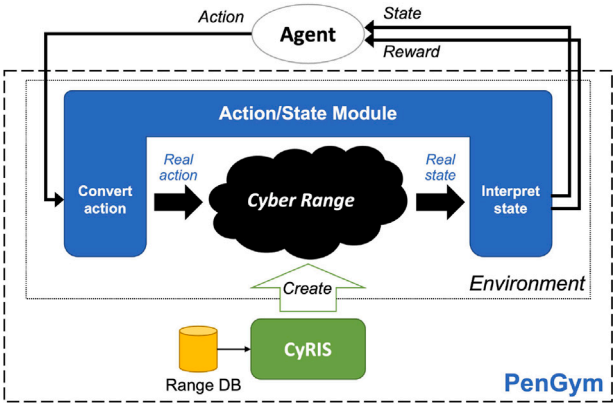


Fig. 3. Overview of extended PenGym architecture.

4. PenGym framework

Given the current limitations and the promising results of our preliminary research, we have expanded PenGym into a fully automated, effective, and realistic training framework for RL pentesting agents. It supports both network traffic discovery and host-based exploitation actions, including all the actions in NASim. PenGym operates in two modes: a simulation mode and an emulation mode. The simulation mode uses a revised simulation environment, NASim(rev.), to simulate actions by checking several logical conditions, implemented as an extension of the NASim framework by fixing several unrealistic errors. The emulation mode employs a realistic environment containing actual pentesting actions and network systems. Additionally, CyRIS is integrated to automate the creation of cyber ranges.

This section introduces first the overall architecture and workflow of PenGym. We then explain the implementation of the entire action space from theoretical and technical perspectives. Additionally, we discuss the optimization mechanisms we introduced to minimize the time of action execution and RL agent training. Finally, we describe the implementation of cyber range description generation.

4.1. Framework overview

PenGym is a framework designed to automatically create and manage realistic environments for training RL agents in pentesting. It provides a network environment that supports various pentesting tasks, allowing RL agents to learn action strategies through interaction experiences.

Fig. 3 shows an overview of PenGym, which consists of two main components. The *Action/State Module* serves as an interface between the agents and the *Cyber Range*, which is used to execute the real actions of the agents. The automatic cyber range creation mechanism is integrated into PenGym by leveraging CyRIS. This paper uses NASim scenario files for the agent training environment and cyber range preparation, stored in the *Range DB* database.

The Action/State Module provides a range of pentesting actions that are executed on cyber range hosts. This module is responsible for handling action execution and state retrieval between agents and the training environment, including two main functions:

- Convert the logical actions selected by the RL agent into executable realistic actions that are executed in the cyber range.
- Interpret the actual results of the actions into the state of the environment and corresponding reward, which are then returned to the agent for further learning.

The process starts with the agent selecting an action from the action space based on the learning policy. This selected action is then converted into a realistic action and executed in the cyber range created by CyRIS. Following action execution, the module interprets the observations and sends them back to the agent, along with a reward. The agent then updates its learning policy to select the next action.

The rewards are defined in the scenario file, with positive values that are to be added or subtracted from the total reward based on the success or failure of each action.

4.2. Action implementation

The action space currently implemented in PenGym covers all the actions in NASim. In NASim, actions mimic the functionality of pentesting tools, such as Nmap (Lyon, 2014). For scan actions, it checks logical conditions and returns predefined values. For host-based exploitation actions (e.g., `e_ssh`), the success is determined based on the existence of the target service or process (e.g., `ssh`), and a predefined success probability for each action.

In contrast, PenGym uses actual actions executed on cyber range hosts. The `python-nmap` library (Norman, 2021) is used to scan and obtain the host information. For host-based exploitation actions, the Metasploit pentesting framework (Maynor, 2011) is utilized to execute the corresponding actions on the target hosts by leveraging the `pymetasploit3` Python library (McInerney, 2020). The PenGym action space is divided into six categories:

1. Service Scan
2. Operating System (OS) Scan
3. Subnet Scan
4. Exploit
5. Process Scan
6. Privilege Escalation

Table 2 provides a summary of the implementation of all actions, showing the method used for each of them, the action description, and the acquired access level for host-exploitation actions (i.e., the available user access permissions after successful exploitation).

4.2.1. Scan actions

Most scan actions utilize `nmap` for scanning and gathering target host information. For the Service Scan and OS Scan, `nmap` employs TCP SYN scan, which sends several network packets to a host and analyzes the response packets to retrieve the running services and OS information of this host.

For the Subnet Scan, `nmap` sends a ping message to each IP address within the specified network range of interconnected subnets to identify active neighboring hosts.

For Process Scan, due to the need for a shell beforehand, the `ps` command is executed directly into this shell to collect the running processes on the target host. Note that the implementation of scan actions was retained from Nguyen. et al. (2024).

4.2.2. Exploit actions

The exploit action category contains five actions, each corresponding to a different service. Each action attacks the host based on the vulnerability of its respective service.

According to the implementation presented in Algorithm 1, depending on the specific service, the corresponding module is called with the required parameters to execute the action. For instance, the `e_smtp` action calls the module `smtp/opensmtpd_mail_from_rce` to gain unauthorized access through the vulnerability of the `smtp` service and returns a shell object that allows shell command execution.

Table 2
Summary of the methods and Metasploit modules used for the PenGym action space implementation.

Actions		Method/ Metasploit module	Description	Access level
Scan Actions	service_scan	nmap scan	Scan the **running services** of the current machine	N/A
	os_scan	nmap scan	Scan the OS type of the current machine	
	process_scan	ps command line	Scan the **running processes** of the current machine	
	subnet_scan	nmap scan	Scan the **connected hosts within the subnet** of the current machine	
Exploit Actions	e_ssh	ssh/ssh_login	Execute a SSH dictionary attack to get the user shell	USER
	e_ftp	ftp/vsftpd_234_backdoor	Send the specific bytes on port 21 to **trigger the** vsf_sysutil_extra() function (CVE-2011-2523)	ROOT
	e_http	http/apache_normalize_path_rce	Exploit an unauthenticated Remote Command Execution vulnerability via the **misconfiguration of the package** (CVE-2021-41773)	USER
	e_samba	samba/is_known_pipename	Exploit the Remote Command Execution vulnerability by **triggering an arbitrary shared library load vulnerability** in Samba (CVE-2017-7494)	ROOT
	e_smtp	smtp/opensmtpd_mail_from_rce	Exploit a **command injection** in the MAIL FROM field to execute a command as the root user (CVE-2020-7247)	ROOT
Privilege Escalation Actions	pe_tomcat	local/cve_2021_4034_pwnkit_lpe_pkexec	Exploit a **bug in the polkit pkexec binary** that relates to how it processes arguments (CVE-2021-4034)	ROOT
	pe_proftpd	ftp/proftpd_133c_backdoor	Exploit a **malicious backdoor** that was added to the ProFTPD download archive (CVE-2015-3306)	ROOT
	pe_cron	local/cron_persistence	Create a cron or crontab entry to **execute a payload**	ROOT

Algorithm 1 Exploit Action

```
Require: host, service
  msfprc ← get_msfprc_client()
  shell ← check_shell_exist()
  if shell exist then
     shell ← get_existed_shell_of_host()
     access_level ← get_host_access_level(shell)
     Return shell, access_level
  else
     module ← get_exploit_module(service)
     result ← execute_exploit_module(module, host)
  end if
  shell ← get_shell(result)
  access_level ← get_host_access_level(shell)
  Return shell, access_level
```

4.2.3. Privilege escalation actions

The privilege escalation action category consists of three actions, each corresponding to a different process. Privilege escalation is achieved by gaining root access to the target system through a vulnerability in the corresponding process. A shell with user access is required in advance to launch the attack and obtain the root shell object as the outcome.

Similar to exploit actions, the attack module and its parameters are changed to be compatible with the specific process. Each module gains unauthorized access to the target through different vulnerabilities, so the configuration of each module is adjusted accordingly. For instance, the pe_cron action calls the local/cron_persistence module to attack the target host and gains root access due to a misconfiguration in the cron process. The pseudocode of the privilege escalation actions is shown in Algorithm 2.

Algorithm 2 Privilege Escalation Action

```
Require: host, process
  msfprc ← get_msfprc_client()
  root_shell ← check_root_shell_exist()
  if root_shell exists then
     root_shell ← get_root_shell()
     access_level ← get_host_access_level(root_shell)
     Return root_shell, access_level
  else
     shell ← get_existed_shell_of_host()
     module ← get_exploit_module(process)
     result ← execute_pe_module(module, host, shell)
  end if
  root_shell ← get_shell(result)
  access_level ← get_host_access_level(root_shell)
  Return root_shell, access_level
```

We note that, for the PenGym implementation, some logical NASim actions in the privilege escalation category were replaced with similar actions due to OS limitations. Thus, the pe_tomcat action in

Table 3
Summary of the implementation and validation of action optimization techniques.

Actions		Initial execution time [s]	Optimized execution time [s]	Optimization Techniques
Scan Actions	service_scan	**15.4799**	**6.6132**	Add ports, arguments: -Pn, -n, -T5 --min-parallel --max-parallel
	os_scan	**9.5054**	**4.5023**	
	process_scan	1.0552	1.0552	
	subnet_scan	**55.9613**	**15.3443**	
Exploit Actions	e_ssh	1.3303	1.3303	Disable the AutoCheck attribute in Metasploit Stop Metasploit job as soon as all the necessary information in a shell is obtained
	e_ftp	10.0423	9.1394	
	e_http	**20.6608**	**6.5873**	
	e_samba	10.9031	9.8926	
	e_smtp	8.1215	6.5839	
Privilege Escalation Actions	pe_tomcat	**219.931**	**13.0525**	
	pe_proftpd	**28.8921**	**20.617**	
	pe_cron	75.6347	68.0839	

NASim was revised to use the pkexec package instead of the tomcat package due to the inability to gain root access directly through a vulnerability in the tomcat process in recent OSes. Moreover, the pe_daclsvc and pe_schtask Window-based actions in NASim are replaced with pe_proftpd and pe_cron. This is because only Linux OS is currently supported in the PenGym cyber range.

4.3. Action optimization

This research implemented two optimization mechanisms to improve the execution performance. The individual action optimization reduces the execution time for a single action, whereas the optimization of training time aims to minimize the total training time by preventing unnecessary repeated execution of actions.

4.3.1. Individual action optimization

We summarize in Table 3 the implementation and validation results of all individual action optimization techniques. For the scan actions, we retained several arguments to minimize the Nmap execution time, including: -Pn, -n, -T5, --min-parallel, and --max-parallel. The ports available in the current scenario are also provided to save time by avoiding scanning unavailable ports.

In this paper, we focus on presenting optimization mechanisms for exploit and privilege escalation actions, which are more complicated and take up most of the execution time in pentesting. Stopping conditions are determined to finish the Metasploit job earlier for these actions. This job will terminate once all shell information is obtained. However, some information is not required for subsequent actions. Stopping the job early can reduce execution time while preserving the functionality of the shell object.

We conducted several tests to demonstrate the advantages of each optimization technique in terms of action execution time. For host-based scanning and exploitation actions, a single host is configured with all services and processes to measure the execution time of these actions. For the subnet scan, which requires more connected hosts and subnets, the test is performed on Host (2,1) in the medium-multi-site scenario (Fig. 5 in Section 6), which has the most connections to subnets among all available networks in PenGym. In the subnet scan, the number of connected subnets can affect the execution time. Therefore, it is essential to consider the condition of connection complexity, as it can clearly show the effectiveness of the optimization.

According to Table 3, the time needed to execute most actions was reduced. The highlighted results show significant enhancements in execution time after applying optimization techniques. For instance, the execution time for the pe_tomcat action has been reduced from nearly 220 s to 13 s, representing a 17x improvement. In complex scenarios where this action is executed on multiple hosts, this reduction substantially impacts the overall execution time. Note that optimization techniques were not applied for process_scan and e_ssh due to the simple command-line execution nature of these actions.

4.3.2. Training time optimization

Regarding optimizing the training stage, we utilized a *host map* dictionary and a *boolean flag* to minimize the execution time of redundant actions. The *host map* dictionary stores the actual observation and related information after the first successful execution of an action, which can then be reused instead of executing the action again. The *boolean flag* is used in conjunction with the *host map* dictionary to mark the state of each host. The flag assists in determining if the re-execution of a specific action is necessary during the training phase, preventing an actual action from being executed multiple times unnecessarily.

In practice, the success or failure of service scans, OS scans, and exploit actions depend on the traffic rules between each subnet. A service scan between two hosts may fail due to a lack of permitted services, and the *host map* dictionary will reuse this result for any future service scans between them. However, if this service scan is initiated from a different host, the actual service scan must be re-executed. Consequently, the *boolean flag* is used to manage the state for the service scan, OS scan, and exploit actions.

To evaluate the effect of the optimization, we calculated the average training time per episode based on a training conducted over 20 episodes in the medium-multi-site scenario for two ql_replay agents that were trained before and after applying the optimization techniques. The average training time per episode was approximately 2.15 h before optimization, compared to 0.05 h after optimization, which represents a reduction of about 43 times. This demonstrates that by avoiding executing actions that were already successful, the optimization helps reduce the training time significantly. We also note that our optimizations reflect real-world situations when a pentester does not repeat successful actions. Hence, using the *host-map* dictionary and *boolean flag* in PenGym considerably lowers the training time while maintaining the realism of the training process.

4.4. Cyber range creation

This section presents the module implemented to support automated cyber range creation by integrating the functionality of CyRIS into PenGym. It covers the use of bridge functionality to enhance the realism of network communication.

CyRIS (Beuran et al., 2018) is designed to facilitate cybersecurity training by automating the creation and management of the required training environment. It uses a text-based representation in YAML format to describe the characteristics of this cyber range, including both environment setup and security content generation. KVM virtualization technology (Habib, 2008) is employed to construct a customized cyber range, including virtual machines with predefined services, processes, and vulnerabilities, all connected following the description file. This aids in preparing and installing the corresponding cyber range instance on a computer and network infrastructure.

4.4.1. Cyber range description generation

In this research, we implemented a PenGym module that transforms the NASim training scenario file into a CyRIS cyber range description

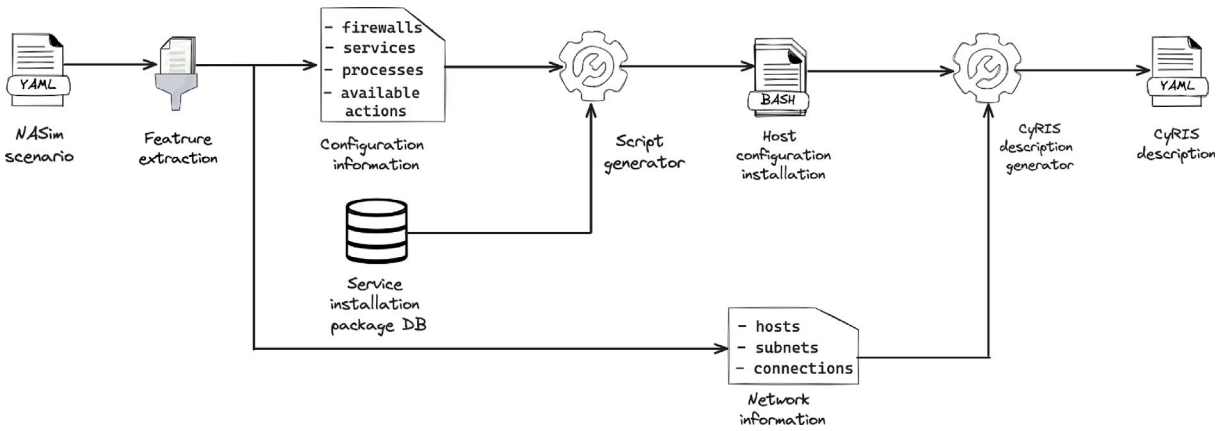


Fig. 4. Processing flow of the CyRIS cyber range description file generation module.

file, thereby automating the cyber range creation process. The purpose is to create a fully configured network at both network and system levels, thereby preparing the training environment for pentesting. Fig. 4 indicates the processing flow of this module, as follows:

- **Information extraction:** In this step, *configuration information* is first extracted from the NASim scenario file, including details about firewalls, services, processes, and host-based exploitation actions. Then, as needed for the last processing step, *network information* is extracted, defining the services or processes available for each host and the subnet connections.
- **Script generation:** The configuration information and the corresponding service installation packages form the input for this step. The *Service Installation Package DB* is an external component that needs to be prepared ahead, containing all the service packages and installation requirements for each service and process. For instance, the `opensmtpd-6.6.1` package is prepared in advance for `smtp` service setup.
  Based on the existing services or processes in a scenario, the corresponding packages and requirements are used to generate a list of *host configuration installation* shell scripts. These scripts include firewall configuration, service and process installation, and vulnerability settings. The shell scripts are subsequently executed to automate the setup procedure.
- **CyRIS description generation:** After preparing the installation scripts, they are combined with the network information from the first step. This information is used to generate a *CyRIS description* file. All cyber range information is output in this phase according to the CyRIS description file structure.

4.4.2. Bridge functionality

Network bridges are used to improve the realism of subnet communication and the effectiveness of the cyber range preparation. They control the network connection and enable interaction between the main server that is running PenGym and the cyber range. PenGym is installed on that server, and the cyber range is also created on it by using CyRIS. All actions executed on the virtual machine originate from the main server. Instead of setting up necessary tools like Metasploit or Nmap on each virtual host, these actions can be executed from the main server using bridge functionality.

Bridges have two main states: on and off. Initially, most bridges are turned off, except for the bridge connecting the cyber range subnets to the virtual Internet cloud. If any host in the cyber range is compromised, the corresponding bridge is turned on. This allows the interaction between the main server and the affected host, as well as connections between its subnet and other subnets. The bridge functionality maintains the simple nature of the cyber range by not installing any external pentesting framework while also reflecting the realism of network communication.

5. Functionality validation

This section discusses the functionality validation of PenGym via detailed results and observations from actions performed in both NASim and PenGym. It also includes validating cyber range creation regarding execution time and environment configuration.

5.1. Action implementation validation

Although we conducted validation experiments for the action space across all available scenarios, we selected the medium-multi-site scenario (see Fig. 5) for explanation purposes. This is because it is a complex scenario in which each host contains multiple services and processes, reflecting the complexity of real-world conditions to demonstrate detailed comparison results. Table 4 summarizes the validation results for this scenario in NASim and PenGym. According to the table, the observations of each action in both environments are equivalent.

Service Scan, OS Scan, and Process Scan: In NASim, the list of services, processes, or OSs corresponding to the current scenario is returned, with `1` and `0` representing `True` and `False`, respectively. Table 4 shows the results for Host (2,1). For instance, for service scan, only the `smtp` service receives a `1` value since it is the only service running on the target host. An equivalent result is produced in PenGym, which returns a list of available services.

Subnet Scan: PenGym returns the list of IP addresses for all hosts within each interconnected subnet of the target host. In NASim, all hosts are returned with a value of `True` for those discovered and `False` for those not yet discovered. We can validate the equivalence between NASim and PenGym observations by mapping these hosts to their associated addresses. For example, a subnet scan on Host (2,1) leads to discovering all the hosts in the network, as that host is connected to all subnets in the scenario.

Exploit and Privilege Escalation: Upon successfully gaining unauthorized access to the target host, a shell object and access level are obtained. In PenGym, this can be identified by executing the `whoami` command line directly through this shell. Table 4 shows again the equivalence between the NASim and PenGym observations.

5.2. Cyber range creation validation

Automated cyber range creation is an important new aspect of this research, using a predefined scenario to create the corresponding cyber range automatically. The duration for creating all scenarios was measured to demonstrate the efficiency of integrating this component into PenGym. Specifically, the creation for medium-multi-site, which is the largest and most complex network scenario in our experiments, took approximately 5390 s. Although this may seem long, note that a

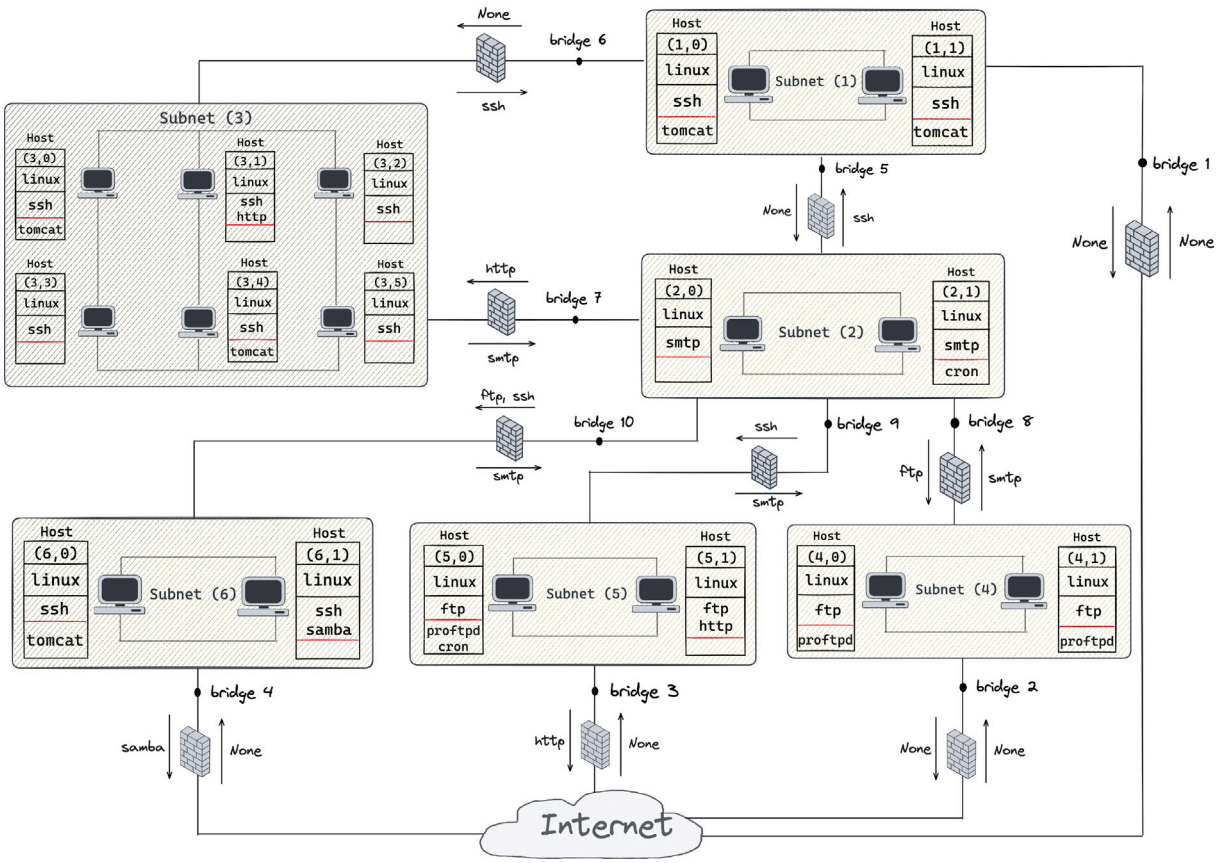


Fig. 5. Cyber range constructed in PenGym based on the medium-multi-site scenario in NASim.

Table 4
The validation of actions between NASim and PenGym.

Actions		Observation	
		NASim	PenGym
Scan Actions	service_scan	'ssh': 0, 'http': 0, 'ftp': 0, 'smtp': 1, 'samba': 0	['smtp']
	os_scan	'linux': 1	['linux']
	process_scan	'tomcat': 0, 'proftpd': 0, 'cron': 1	['cron']
	subnet_scan	(1, 0): True, (1, 1): True... (6, 0): True, (6, 1): True	['44.1.5.2', '44.1.5.3'... '44.1.10.4', '44.1.10.5']
Exploit Actions	e_ssh	access=USER	
	e_http		
	e_ftp	access=ROOT	
	e_samba		
	e_smtp		
Privilege Escalation Actions	pe_tomcat	access=ROOT	
	pe_proftpd		
	pe_cron		

cyber range is created only once during experimentation, and around 50% of the creation time comes from the package installation process.

The cyber range configurations are verified on each host by running a shell command to check the currently active services or processes. Network traffic on each host is validated using the `iptables` command line. For instance, the services and processes on Host (5, 0) are validated for the medium-multi-site scenario by executing the command "`ps -aux | grep -e ftp -e ssh -e proftpd -e cron`". The output showed the actual running services and processes on this host, such as the `ftp` service, along with the `proftpd` and `cron` processes. To ensure no undesired services are running on a host, the other services and processes (i.e., `ssh`) are also checked as part of the validation procedure.

The validation process assessed the correct creation of the cyber range concerning the predefined scenario, with completely automated host and network configuration.

6. Experiment results

We conducted a series of experiments to demonstrate the feasibility and effectiveness of the PenGym in terms of time performance, stability, and adaptability of trained agents when dealing with various complex

Table 5
Overview of all the experiment scenarios in PenGym.

Network Size	Scenario	Number of subnets	Number of hosts	Action space size	Optimal step count
Tiny	**tiny**[a]	**3**	**3**	**18**	**6**
	tiny-hard	3	3	27	5
	tiny-small	4	5	45	7
Small	small-honeypot	4	8	72	7
	small-linear[a]	**6**	**8**	**72**	**12**
Medium	medium	5	16	192	7
	medium-single-site	1	16	192	4
	medium-multi-site[a]	**6**	**16**	**192**	**7**

[a] These scenarios were selected for detailed experiments.

pentesting scenarios. We selected NASim for comparison to highlight the key differences between PenGym as an emulation approach and NASim as a simulation approach. In addition, we used two versions of the NASim. The first one, NASim, is the original NASim environment, and the second one, NASim(rev.), is the revised version of NASim for which we corrected some logical errors in the original version, as it will be explained in Section 7. Thus, the RL agents were trained and tested using NASim, NASim(rev.), and PenGym.

6.1. Experiment scenarios

We covered all scenarios available in NASim, categorized based on the network size as *tiny*, *small*, and *medium*. Table 5 outlines the attributes of all scenarios, including the number of subnets, number of hosts, action space size, and optimal step count to reach the pentesting goal.

Due to space limitations, only medium-multi-site, the largest scenario available in NASim, is selected for visualization, as shown in Fig. 5. This scenario comprises 16 hosts divided into 6 subnets, with hosts (2, 1) and (3, 4) being the pentesting goals. Each host contains one or multiple services and processes. Some hosts may lack either element, which increases the complexity of the network. All actions are available in this scenario. Network traffic restrictions between each subnet were implemented using the `iptables` Linux firewall configuration utility. All the experiments were conducted on a dual 12-core 2.2 GHz Intel(R) Xeon(R) Silver 4214 CPU server with 64 GB RAM.

6.2. Preliminary experiments

We conducted a set of preliminary experiments in all scenarios to demonstrate the overall capability of the agents trained in the PenGym and NASim environments. These experiments also aid in deciding the representative scenario for each network level to conduct the detailed experiments.

Experiment conditions. To decide the algorithms used for the agents in this experiment, in the medium-multi-site network scenario, we used the tabular, epsilon-greedy Q-learning with experience replay (`ql_replay`) (Sutton and Barto, 2018), Q-learning (`ql`) (Watkins and Dayan, 1992), and DQN (Mnih et al., 2015) algorithms agents, available in NASim, for training. These agents were trained in the PenGym environment. Our initial results showed that the `ql_replay` agent took about 6 h to converge within 300 episodes, while the `ql` agent achieved stability within 4000 episodes in approximately 2 h. Despite requiring more episodes, the training time of the `ql` agent was reasonably efficient compared to the `ql_replay` agent. However, the DQN agent using default settings could not converge due to its simple architecture, which consists of only two hidden layers with 64 nodes each and uses the default Adam optimization.

Based on these results, we selected the `ql` agent for the preliminary training experiments across all scenarios. For this, we independently trained 5 `ql` agents for 4000 episodes and tested each agent once in the PenGym and NASim (rev.) environments across all scenarios. Moreover, we selected medium-multi-site, small-linear, and tiny scenarios for the same experiments with the NASim environment.

Experiment results. Table 6 summarizes the results of the preliminary experiments. The training time in all scenarios was measured, and the average ratio between the PenGym and NASim(rev.) environments was calculated based on the network type. The results indicated that the training time ratio decreases as network complexity increases. It reduced from approximately 6.0 to 1.7 as the network grew. The training time in both environments can be comparable when dealing with complex network scenarios. This leads to a reasonable time, given the complexity of real-life networks.

Regarding adaptability, the agents trained in NASim performed well in the simulation environment (NASim). However, their performance declined in the realistic environment (PenGym). In more complex scenarios, such as small-linear, an increase in the number of steps indicated a significant gap in reaching the goal between both environments. Particularly in the medium-multi-site scenario, the agents completely failed during PenGym environment testing by reaching the imposed step limit of 2000 without reaching the goal.

Moreover, the NASim(rev.) and PenGym produced comparable results, with a small gap in the testing steps across all scenarios. However, the step differences pointed out that agents trained in PenGym had more stable performance than those trained in the NASim(rev.). The step difference for PenGym-trained agents ranges from 0.4 to 1, whereas it is between 0 and 3.8 for the NASim(rev.) environment.

6.3. Detailed experiments

We then carried out a series of detailed experiments to showcase the effectiveness of PenGym. The preliminary results in Section 6.2 showed that the complexity of the scenarios impacts the training performance and convergence of the RL agents. In these experiments, we chose three scenarios, tiny, small-linear, medium-multi-site, representing three different complexity levels, for detailed training and testing purposes. As before, we trained and tested the RL agents using NASim, NASim(rev.), and PenGym environments.

6.3.1. Agent training

We will begin by discussing the agent training procedure, starting with the experiment conditions, followed by the results we obtained.

Experiment conditions. For each scenario, we trained 10 agents independently for the NASim, NASim(rev.), and PenGym environments.

Table 6
Results of preliminary experiments conducted for all the available scenarios (NASim* refers to either NASim or NASim(rev.), according to the simulator variant used for training).

Network	Scenario	Training results			Testing steps		
		Environment	Training time [s]	Ratio[a]	NASim* environment	PenGym environment	Step difference
Tiny	tiny	NASim	33.721436	5.958373	**6.6**	**7**	0.4
		NASim(rev.)	34.529200		7.4	7	0.4
		PenGym	159.933349		7.6	7	0.6
	tiny-hard	NASim(rev.)	34.529200		6.6	7.8	1.2
		PenGym	246.850449		5.4	5.4	0
	tiny-small	NASim(rev.)	77.674751		12	8.8	3.2
		PenGym	473.369863		8.8	9.5	0.7
Small	small-honeypot	NASim(rev.)	151.095278	4.548211	8.8	8.8	0
		PenGym	1019.356081		10.4	11.8	1.4
	small-linear	NASim	223.226071		**21.2**	**213.8**	192.6
		NASim(rev.)	373.258370		15.2	14.8	0.4
		PenGym	877.148658		14.8	15.8	1
Medium	medium	NASim(rev.)	3668.875337	1.688260	12.8	16.6	3.8
		PenGym	6438.764622		14.2	14	0.2
	medium-single-site	NASim(rev.)	2630.687669		6.6	6.4	0.2
		PenGym	4158.419250		6.6	6.2	0.4
	medium-multi-site	NASim	4890.494927		**24.6**	**2000**	1975.4
		NASim(rev.)	4348.470149		26	24.2	1.8
		PenGym	7518.836585		24.6	24	0.6

[a] Average ratio of training time values in PenGym versus NASim(rev.) for a given network type.

Specifically, we trained 5 agents using the `ql_replay` and 5 agents using the `ql` algorithms that are mentioned in Section 6.2. These algorithms were used due to their ability to converge and maintain stability during the training phase, as shown in our initial experiments. The DQN algorithm was excluded due to the inability to converge the learning strategy of the agent. For the `ql_replay` algorithm, we trained each agent with 300 episodes, and for the `ql` algorithm, we used 4,000 episodes to achieve the convergence for each algorithm. The episode gap between the two algorithms arises from their complexity. While the `ql_replay` agent employs experience replay to accelerate convergence, the `ql` agent uses a simple Q-learning algorithm.

Note that during training, we reset the environment after each episode in NASim, NASim(rev.), and PenGym to maintain consistency. This resets the State and Observation objects and the step count to their initial values.

For PenGym, we reset two key elements to maintain agent realism: the boolean flag and all network bridges. After the first episode, most host and topology information is explored and stored for reuse. However, some actions may not have been executed successfully yet. Resetting the boolean flag allows for re-executing necessary actions while resetting bridges ensures realistic subnet connections.

Upon completing an agent's training in PenGym, we also cleaned up the Metasploit session, reset all host map values, and reset the firewalls for all hosts. This ensures consistent conditions for each new training.

Experiment results. Fig. 6 shows the average rewards obtained during the training of 5 `ql` agents and 5 `ql_replay` agents across three scenarios for the NASim, NASim(rev.) and PenGym environments. According to the results, the PenGym framework has demonstrated comparable performance in training agents in terms of rewards compared to NASim and NASim(rev.), using both types of agents. However, in a complex scenario such as `medium-multi-site`, the training results still oscillated more in both NASim and NASim(rev.) environments. Although the results from the NASim environment are better than others in the `medium-multi-site` scenarios, this is only due to the incorrect logical model that enables the agent to reach the goal in fewer steps, ignoring some permission rules.

Regarding the training time, the difference decreased as the network and RL algorithms became more complex. Notably, in the most complicated scenario, `medium-multi-site`, PenGym averaged nearly 17,000 s compared to 14,000 s for NASim(rev.), making the training time comparable between two environments.

Training time composition analysis. Based on experiments of training a QLearning Replay agent for 300 episodes when using the largest scenario in both the NASim(rev.) and PenGym environments, we calculated the proportion of the time needed to reset the environment, update the agent policy, and execute actions in the environment compared to the total training time. Our results showed that for the NASim(rev.) environment, 98% of the training time came from agent policy updating, while the rest of the time was used for environment resetting and action execution. Consequently, the action execution time in the simulation environment is minimal, with each action taking only a few milliseconds due to simple condition checks, and most of the training time is used to update the agent learning strategies.

On the other hand, in the PenGym environment, agent policy updating accounted for only 58% of the training time, whereas action execution time was 40%, and environment resetting took 2% of the time. Even though the proportion of agent policy updating time compared to total training time differs between the two environments, the ratio between PenGym and NASim(rev.) was comparable, around 1.1. This is because the policy update time depends on the number of attack steps. When the training performance of the agents is equivalent between the two environments, the time for updating their policies is also similar.

We conclude that, even though the training time in PenGym is somewhat longer than in NASim(rev.), it remains within a reasonable limit and is comparable in complex scenarios. The difference in total training time between the PenGym and NASim(rev.) environments can be attributed to the longer action execution times in PenGym. This is caused by the fact that the actions are executed in an actual environment, meaning that more time is spent exploring the environment in PenGym compared to NASim(rev.)

6.3.2. Agent testing

Next, we evaluated the performance of the trained agents in the NASim, NASim(rev.), and PenGym environments.

Experiment conditions. We conducted 10 testing experiments in each scenario within either the NASim or NASim(rev.) and PenGym environments. To illustrate the advantages of using the RL approach in pentesting, the brute force agent available in NASim was also included. This agent uses the *"try all possible actions to reach the goal"* approach, and was executed for 10 trials as well. While both random and brute

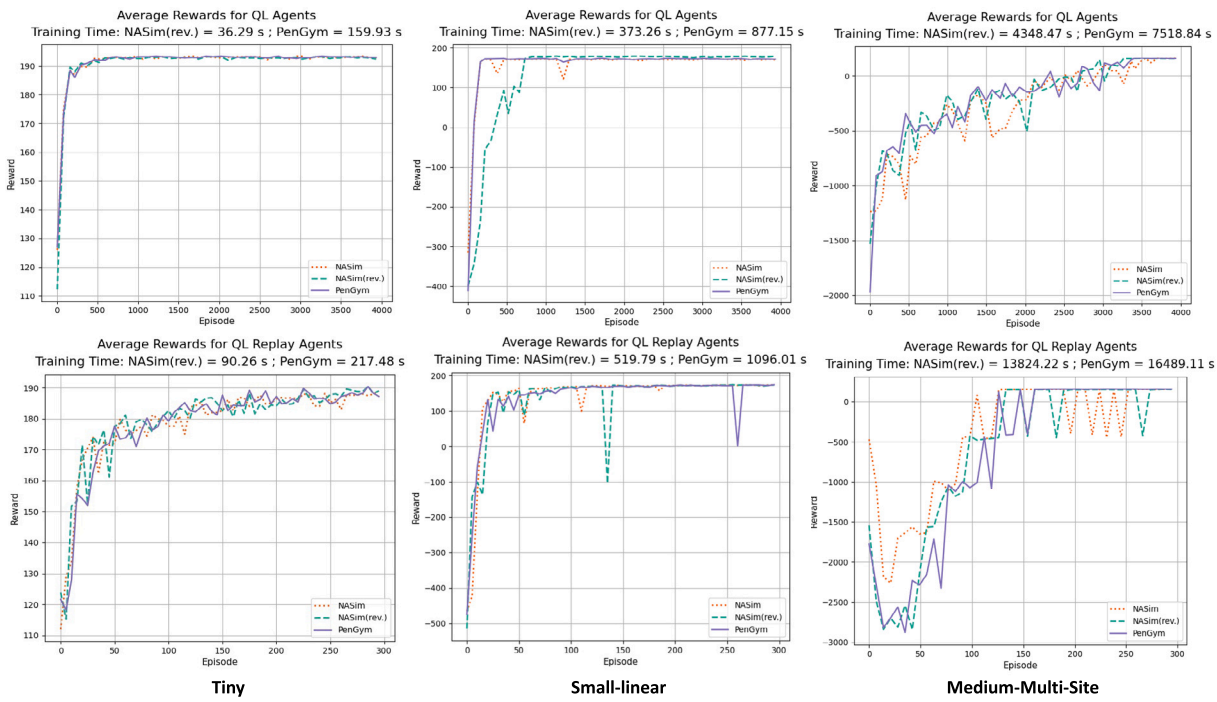


Fig. 6. Average training reward versus episode number for the QLearning and QLearning Replay agents trained in the NASim, NASim(rev.), and PenGym environments for the three representative experiment scenarios.

force agents are available for conducting experiments, we only chose to use the brute force agent since it outperforms the random agent in our trial tests.

Result overview. Fig. 7 shows the average attack steps of both the `ql` and `ql_replay` agents, trained on NASim, NASim(rev.), and PenGym then tested in these environments. Figs. 7(a), 7(b), and 7(c) visualize the testing results using the tiny, small-linear, and medium-multi-site scenarios, respectively. The results of the 10 trials were averaged to derive the final performance values for the agents in each environment. The error bars were also added to visualize the standard deviation values for assessing the performance consistency and stability of the agents across different trials.

The results we presented demonstrate that both the `ql` and `ql_replay` agents, when trained in PenGym, successfully achieved the pentesting goals across three scenarios. They showed stable performance with minor variation in both the simulation (NASim and NASim(rev.)) and emulation (PenGym) environments.

However, agents trained in the NASim performed poorly when tested in PenGym. Thus, they only achieved the goals in simple scenarios, like tiny, and became inapplicable in PenGym when faced with more complex scenarios, such as small-linear and medium-multi-site. While these agents achieved the pentesting goal in some trials, they required more steps than the optimal steps listed in Table 5, demonstrating a significant disparity in the number of required steps. Notably, the agents mostly failed in the largest scenario, exceeding the 2000 step limit. Moreover, the error bars in Figs. 7(b) and 7(c) indicate the instability of agents trained in the NASim environment, as suggested by the higher standard deviation. This is because the agents trained in the NASim environment mostly failed, and had a low success rate in reaching the pentesting goals.

On the other hand, since NASim(rev.) corrects the logical modeling issues in NASim, the performance of agents trained in PenGym and the NASim(rev.) is similar in the tiny and small-linear scenarios, with only a minor difference in their averages. This suggests that agents trained in a correctly logical simulation environment can achieve results comparable to those in a realistic environment.

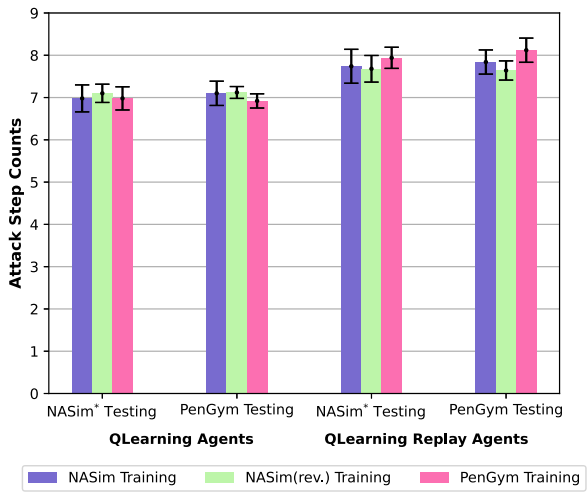
Detailed results. Table 7 details the testing results in medium-multi-site. This table shows that most agents trained in NASim cannot be applied successfully in PenGym, as evidenced by the low success rates of `ql` and `ql_replay` agents, at 0.02 and 0.0 respectively. Despite the agents trained in NASim(rev.) achieving performance comparable to those trained in PenGym, the latter slightly outperformed the former. This is proved when testing `ql_replay` agents trained in NASim(rev.) and PenGym using a realistic environment; the average step difference is around 8 (22.86 steps compared to 30.9 steps). This discrepancy could be due to yet-undiscovered *niche* issues in the NASim(rev.) that affect the performance of agents. Thus, while simulation environments for training pentesting agents have shown promising results in recent years, these results may not be reliable enough for deployment in real-world infrastructure.

Additionally, the results showed that using brute force, a traditional method in pentesting, was less effective than using RL-based agents. From the experiments, the number of steps required significantly increased as the scenario became more complicated. Specifically, it took 681 steps to reach the goal in the medium-multi-site scenario.

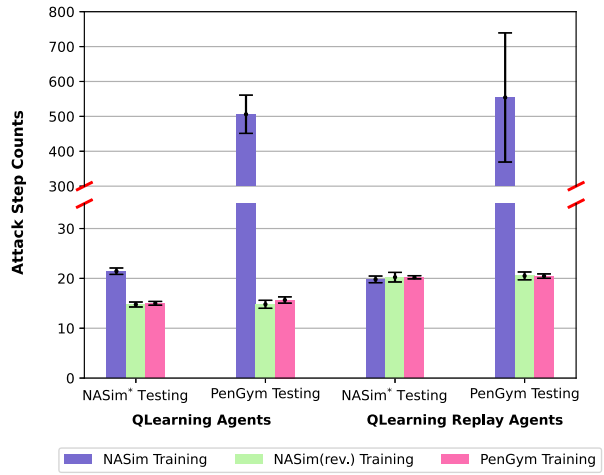
One conceivable drawback of PenGym is its longer execution time compared to the simulation environment during the testing process, as actions are physically executed in the network. Nevertheless, the training time tends to be comparable in more complex scenarios, so we consider the gap in testing time a reasonable trade-off for the reliability of agents trained in a realistic environment.

7. Discussion

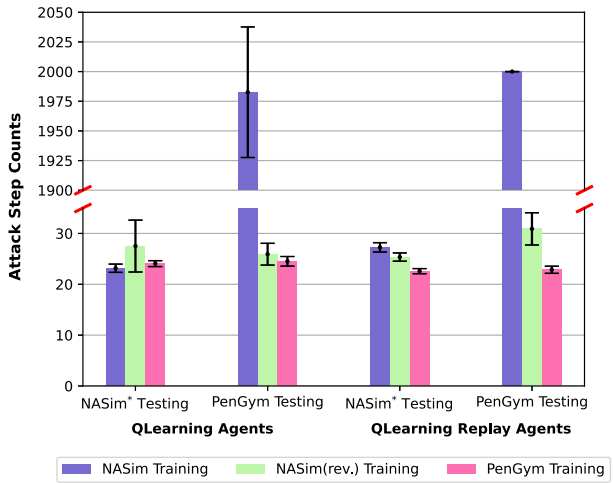
Table 8 presents the differences between PenGym and NASim regarding environment configuration, observation, and functionality problems. Since PenGym supports only the Linux OS, some actions were replaced to be compatible with this environment. For instance, the `'schtask'` process was replaced with the `'cron'` process. These changes were made to ensure consistency between the NASim and PenGym environments, as both were created using the same scenario. Moreover, the observation of exploit actions between NASim and PenGym are different. In NASim, if the exploit action is successful,



(a) Experiment results for the tiny scenario.



(b) Experiment results for the small-linear scenario.



(c) Experiment results for the medium-multi-site scenario.

Fig. 7. Average number of attack steps for the trained QLearning and QLearning Replay agents when tested in the NASim* and PenGym environments for the three representative experiment scenarios (NASim* refers to either NASim or NASim(rev.), according to the simulator variant used for training).

Table 7
Detailed experiment results for the QLearning (`ql`) and QLearning Replay (`ql_replay`) agents trained and tested in the NASim, NASim(rev.) and PenGym environments using the `medium-multi-site` scenario.

Scenario	Algorithm	Training	Testing	Success rate	Avg. Steps	Avg. exec. time [s]	Std. Dev.
Medium-Multi-Site (Medium Network)	QLearning	NASim	NASim	1.0	23.16	0.023	0.810
			PenGym	**0.02**	**1982.60**	173.735	55.024
		NASim(rev.)	NASim(rev.)	1.0	27.52	0.033	5.104
			PenGym	1.0	25.92	470.216	2.142
		PenGym	NASim(rev.)	1.0	24.06	0.025	0.582
			PenGym	1.0	24.52	416.788	0.944
	QLearning Replay	NASim	NASim	1.0	27.26	0.026	0.919
			PenGym	**0.0**	**2000.00**	141.467	0.000
		NASim(rev.)	NASim(rev.)	1.0	25.36	0.026	0.810
			PenGym	1.0	30.90	356.510	3.169
		PenGym	NASim(rev.)	1.0	22.58	0.023	0.503
			PenGym	1.0	22.86	356.684	0.699
	Brute force	N/A	NASim(rev.)	1.0	681.00	0.0517	0.000
			PenGym	1.0	681.00	2545.606	0.000

the observation returns the current state of the network, including all services and OS information, even without prior scan actions. This creates inconsistency from a realistic perspective. The exploit action does not send packages to scan the services or OSes, so this information cannot be obtained without scan actions. To make the observation consistent between two environments, in PenGym, we assumed that

Table 8
Summary of differences between the PenGym and NASim environments.

Category	Criteria	NASim	PenGym	Change location
Host Configuration	OS	Windows, Linux	Linux	PenGym scenario
	Process	`daclsvc`	`proftpd`	
		`schtask`	`cron`	
Acions	Probability	Not fixed (e.g., `e_ssh: 0.9`, `e_ftp: 0.6`)	`0.999999` (exploit actions)	
	`e_smtp`	Access Level: **USER**	Access Level: **ROOT**	
	`pe_tomcat`	Simulate **`tomcat`** based action	Replace by **`pe_pkexec`** action	PenGym scenario; PenGym code
	`pe_daclsvc`	Simulate **`daclsvc`** based action	Replace by **`pe_proftpd`** action	
	`pe_schtask`	Simulate **`schtask`** based action	Replace by **`pe_cron`** action	
Observation	Exploit actions	The services and OS information is included in the observation without prior scan actions	Only the explored services and OS information is included in the observation	PenGym code
Functionality Bugs	The firewall does not affect the exploit action from the Internet to the connected subnet	Exploits based on services not allowed by the firewall can be executed from the Internet to hosts within connected subnets	Check the permission services between the Internet and connected subnets	Original NASim code
	The Service Scan and OS Scan do not take firewall into account	Services or OSes not allowed by the firewall can be detected between source and destination subnets in service scan	Filter the permission between source and destination subnets	PenGym code (NASim mode)
	The Remote Action (Exploit) can be executed on its target host	The remote action can be executed on host itself	Check remotely of the source address when executing remote actions	Original NASim code

when the exploit action is successful and returns a shell object, the services and OS information are scanned using this shell.

Regarding functionality differences, we discovered some unexpected logical errors in NASim and corrected them to be NASim(rev.), ensuring the behavior is comparable to PenGym. They came from uncovered situations when modeling the action, mainly when dealing with complex scenarios that require complicated conditional logic. This complexity of scenarios increased the chances of encountering conditions that we could not manually cover. Specifically, in the NASim environment, an exploit action from the Internet to a connected host will succeed even if no traffic is allowed between them. However, this action cannot occur in real life if the firewall blocks all transactions. For example, the `e_ftp` exploit action from the Internet to Host (4, 1) in the medium-multi-site scenario would succeed in the NASim environment. However, it would fail in the PenGym environment because the exploit module cannot connect to the `ftp` port due to the blocking rule of the firewall.

As another issue, both the OS scan and service scan actions succeed in the NASim environment, even when a firewall blocks the service. This creates an unrealistic outcome since the Nmap scan function will send the TCP packets to the system and determine the running services based on the returned packets. Therefore, if a service is blocked, there should be no packet transmission. Hence, the results from the simulation environment conflict with the real-world conditions, making the trained agents ineffective when deployed in real-world infrastructure.

Advantages and disadvantages of PenGym. PenGym makes it possible to train RL agents via actual attacks on a network that contains actual hosts running real services, processes, vulnerabilities, and firewall rules that are configured inside these hosts using `iptables`. Thus, the success or failure of an action depends on the actual network conditions and the specific vulnerabilities of the hosts.

This means that using an emulation environment eliminates the need to model each action's success conditions using probabilistic attributes associated with executing specific actions. Such conditions are difficult to model in complex scenarios where each host runs multiple services or processes, but only a few services are allowed through the firewall. Moreover, using nested conditions may not cover all the situations encountered in real-world networks.

When using simple scenarios, the agents trained in simulation and emulation environments yield comparable results. However, the training time in the simulation environment is significantly lower than in the emulation environment, emphasizing a disadvantage of emulation.

To overcome this disadvantage, we have implemented optimization techniques in PenGym to minimize the execution time for each action and the overall training time. As shown in the experiment results in Section 6.3.2, after applying these optimizations, the training time becomes comparable between simulation and emulation approaches when training in the most extensive scenario and the more complex Q-learning algorithm.

For simple scenarios, training agents in a simulation environment and validating them in emulation is a promising approach, with significantly shorter training times. However, this requires to ensure that the simulation environment correctly models the real one. This is difficult to achieve for complex scenarios due to the potential existence of niche conditions arising from scenario complexity.

However, for agent deployment on real network infrastructure, more complex scenarios are needed to reflect the intricacies of actual networks accurately. When dealing with these complex scenarios, although simulation-based training may fail due to logical issues, the emulation environment always provides reliable results in agent training, avoids unexpected situations, and achieves training times closer to simulation-based environments.

8. Conclusion

In this paper, we have presented the PenGym framework, which fully supports the creation of realistic training environments for RL pentesting agents, accommodating diverse network scenarios of varying complexity. By enabling the execution of actual actions based on real observations, PenGym provides a reliable and efficient training environment for pentesting RL agents. This eliminates the need for action modeling, thus resulting in a more accurate representation of network and security dynamics than simulation-based environments.

We presented several experiments to emphasize the effectiveness of our approach. For the smallest scenario, agents trained in NASim, NASim(rev.), and PenGym environments achieved equivalent results when testing in both simulation and emulation environments. For

the mid-size scenario, simulation-related issues occurred. Although the agents trained in the NASim environment performed well in the simulation environment, they were ineffective when testing in the emulation environment, showing a high variation and large attack step counts. In contrast, with a more accurate logical model, the NASim(rev.) environment yielded testing results comparable to PenGym in both the simulation and emulation environments.

For the largest scenario, agents trained in NASim mostly failed when testing in PenGym environments, with very low success rates of the `ql` and `ql_replay` agents. The effectiveness of PenGym was clearly shown in these experiments. Although agents trained in NASim(rev.) reached the pentesting goal with reasonable attack steps across all test trials, their performance was inconsistent compared to PenGym, which had a higher stability. The results also indicate that agents trained in PenGym slightly outperformed those trained in NASim(rev.) for all test trials in both simulation and emulation environments, with average step differences ranging from 1.4 to 8 steps. Thus, using PenGym for a realistic training environment enhances the applicability of the trained agents when deployed in a real-world infrastructure.

Due to its cyber range execution, PenGym requires more training time than the NASim(rev.) environment. However, it maintains a reasonable training duration, especially in more complex scenarios. For the largest scenario, and most intricate RL algorithm, PenGym training takes around 17,000 s compared to 14,000 s in NASim, with a ratio of roughly 1.2.

The PenGym framework was released as open source on GitHub (https://github.com/cyb3rlab/PenGym), with full support for all the presented scenarios.

As future work, we aim to propose a realistic automatic scenario generator. Currently, the scenarios used in this study are based on those manually created by the NASim developer. However, a method to automatically generate scenarios resembling real-world infrastructures would enhance the adaptability of the pentesting RL agents trained on them.

CRediT authorship contribution statement

Huynh Phuong Thanh Nguyen: Writing – original draft, Validation, Methodology. **Kento Hasegawa:** Writing – review & editing, Supervision. **Kazuhide Fukushima:** Supervision. **Razvan Beuran:** Writing – review & editing, Supervision, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

We want to thank Mr. Zhi Chen for his assistance with the initial implementation of the PenGym scan actions.

References

Beuran, R., Tang, D., Pham, C., Chinen, K., Tan, Y., Shinoda, Y., 2018. Integrated framework for hands-on cybersecurity training: CyTrONE. Comput. Secur. 78C, 43–59.

Chaudhary, S., O'Brien, A., Xu, S., 2020. Automated post-breach penetration testing through reinforcement learning. In: 2020 IEEE Conference on Communications and Network Security. CNS, IEEE, pp. 1–2.

Furfaro, A., Piccolo, A., Parise, A., Argento, L., Sacca, D., 2018. A cloud-based platform for the emulation of complex cybersecurity scenarios. Future Gener. Comput. Syst. 89, 791–803.

Ghanem, M.C., Chen, T.M., 2018. Reinforcement learning for intelligent penetration testing. In: 2018 Second World Conference on Smart Trends in Systems, Security and Sustainability. WorldS4, IEEE, pp. 185–192.

Ghanem, M.C., Chen, T.M., 2019. Reinforcement learning for efficient network penetration testing. Information 11 (1), 6.

Habib, I., 2008. Virtualization with KVM. Linux J. 2008 (166), 8.

Janisch, J., Pevný, T., Lisý, V., 2024. NASimEmu: Network attack simulator & emulator for training agents generalizing to novel scenarios. In: Computer Security. ESORICS 2023 International Workshops. Springer Nature Switzerland, Cham, pp. 589–608.

Li, L., El Rami, J.-P.S., Taylor, A., Rao, J.H., Kunz, T., 2022. Enabling a network AI gym for autonomous cyber agents. In: 2022 International Conference on Computational Science and Computational Intelligence. CSCI, IEEE, pp. 172–177.

Li, Q., Hu, M., Hao, H., Zhang, M., Li, Y., 2023. INNES: An intelligent network penetration testing model based on deep reinforcement learning. Appl. Intell. 53 (22), 27110–27127.

Lyon, G., 2014. Nmap security scanner. https://nmap.org/.

Maynor, D., 2011. Metasploit Toolkit for Penetration Testing, Exploit Development, and Vulnerability Research. Syngess Publishing, Elsevier.

McInerney, D., 2020. Pymetasploit3. https://pypi.org/project/pymetasploit3/.

Microsoft Defender Research Team, 2021. CyberBattleSim. GitHub, https://github.com/microsoft/cyberbattlesim, Created by Christian Seifert, Michael Betser, William Blum, James Bono, Kate Farris, Emily Goren, Justin Grana, Kristian Holsheimer, Brandon Marken, Joshua Neil, Nicole Nichols, Jugal Parikh, Haoran Wei.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al., 2015. Human-level control through deep reinforcement learning. Nature 518 (7540), 529–533.

Molina-Markham, A., Miniter, C., Powell, B., Ridley, A., 2021. Network environment design for autonomous cyberdefense. arXiv:2103.07583.

Nguyen., T., Chen., Z., Hasegawa., K., Fukushima., K., Beuran., R., 2024. PenGym: Pentesting training framework for reinforcement learning agents. In: Proceedings of the 10th International Conference on Information Systems Security and Privacy - ICISSP. SciTePress, INSTICC, pp. 498–509.

Nguyen, T.T., Reddi, V.J., 2021. Deep reinforcement learning for cyber security. IEEE Trans. Neural Netw. Learn. Syst. 34 (8), 3779–3795.

Norman, A., 2021. Python-Nmap. https://pypi.org/project/python-nmap/.

Pozdniakov, K., Alonso, E., Stankovic, V., Tam, K., Jones, K., 2020. Smart Security Audit: Reinforcement Learning with a Deep Neural Network Approximator. In: 2020 International Conference on Cyber Situational Awareness, Data Analytics and Assessment. CyberSA, pp. 1–8.

Sarraute, C., Buffet, O., Hoffmann, J., 2012. POMDPs make better hackers: Accounting for uncertainty in penetration testing. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 26, No. 1. pp. 1816–1824.

Schoonover, K., Michalak, E., Harris, S., Gausmann, A., Reinbolt, H., Tauritz, D.R., Rawlings, C., Pope, A.S., 2018. Galaxy: a network emulation framework for cybersecurity. In: 11th USENIX Workshop on Cyber Security Experimentation and Test. CSET 18, pp. 1–8.

Schwartz, J., Kurniawati, H., 2019. Autonomous penetration testing using reinforcement learning. arXiv:1905.05965.

Standen, M., Lucas, M., Bowman, D., Richer, T.J., Kim, J., Marriott, D., 2021. CybORG: A gym for the development of autonomous cyber agents. In: Proceedings of the 1st International Workshop on Adaptive Cyber Defense. pp. 1–7, URL https://arxiv.org/html/2108.08476v1.

Stefinko, Y., Piskozub, A., Banakh, R., 2016. Manual and automated penetration testing. Benefits and drawbacks. Modern tendency. In: 2016 13th International Conference on Modern Problems of Radio Engineering, Telecommunications and Computer Science. TCSET, IEEE, pp. 488–491.

Sutton, R.S., Barto, A.G., 2018. Reinforcement Learning: An Introduction. MIT Press.

The MITRE Corporation, 2018. BRAWL. GitHub, https://github.com/mitre/brawl-public-game-001.

Watkins, C.J., Dayan, P., 1992. Q-learning. Mach. Learn. 8, 279–292.

Zennaro, F.M., Erdődi, L., 2023. Modelling penetration testing with reinforcement learning using capture-the-flag challenges: Trade-offs between model-free learning and a priori knowledge. IET Inf. Secur. 17 (3), 441–457.

Huynh Phuong Thanh Nguyen received her B.Eng. Computer Science and Engineering degree from International University, Vietnam National University, Vietnam, in 2022. She is currently pursuing a Master's degree in Information Science from Japan Advanced Institute of Science and Technology (JAIST), Ishikawa, Japan. Her current research topics are network security and AI-based automation of penetration testing procedures.

Kento Hasegawa received his B.Eng., M.Eng., and Dr.Eng. degrees from Waseda University in 2016, 2017, and 2020, respectively, all in computer science and communications engineering. He is presently working for KDDI Research, Inc, and his research interests include hardware security and machine learning. He is a member of the Institute of Electronics, Information and Communication Engineers (IEICE) and the Information Processing Society of Japan (IPSJ).

Kazuhide Fukushima received his M.E. in Information Engineering from Kyushu University, Japan, in 2004. He joined KDDI and has been engaged in research on post-quantum cryptography, cryptographic protocols, and identification technologies. He is currently a senior manager at the Information Security Laboratory of KDDI Research, Inc. He received his Doctorate in Engineering from Kyushu University in 2009. He received the IEICE Young Engineer Award in 2012. He is a member of the Institute of Electronics, Information and Communication Engineers (IEICE) and the Information Processing Society of Japan (IPSJ).

Razvan Beuran received the B.Sc., M.Sc., and Ph.D. degrees from Politehnica University, Bucharest, Romania, in 1999, 2000, and 2004, respectively; the Ph.D. degree was delivered jointly with Jean Monnet University, Saint Etienne, France. From 2006 to 2015, he was with the National Institute of Information and Communications Technology, Hokuriku StarBED Technology Center, Japan. Since 2015 he is Associate Professor with the Japan Advanced Institute of Science and Technology (JAIST), Ishikawa, Japan. His current research topics are cybersecurity, IoT/CPS security, trustworthiness, and cybersecurity training. He is a senior member of IEEE, and a member of ACM and IPSJ.